Universidad Rafael Landívar

Ciencia de Datos · Segundo semestre 2026

Ing. Max Cerna

Proyecto 2 (Fase 1)

Red Metropolitana

¿Quién deja de usar la red?

Integrantes:

Yasmely Dubraska Fariñas Aldana 1329223

Jose Rodrigo Herrera Muralles 1042619

Walter José Herrera Salvatierra 1109722

28 de octubre de 2026

Resumen

En esta fase usamos la plataforma del Proyecto 1 para responder una pregunta concreta: con lo que sabemos de un usuario hasta hoy, ¿qué tan probable es que deje de usar la red en los próximos 30 días? Cargamos los datos nuevos (90 días, del 1 de junio al 29 de agosto de 2026) con el mismo flujo, construimos la población y la etiqueta, armamos la tabla de features con la fecha de corte como parámetro, revisamos que no haya fuga con la prueba de recorte y comparamos tres líneas base con una regresión logística regularizada (L2, L1 y Elastic Net) con validación cruzada en el corte A. La etiqueta del corte B no se usó en esta fase.

Qué medimos	**Resultado**
Carga desde cero	9 archivos, 3,405,575 filas, 0 diferencias con los conteos esperados
Corte A (entrenamiento)	7,095 / 59,491 = 0.1193 positivos
Corte B (test, sin usar en esta fase)	6,387 / 58,113 = 0.1099 positivos
Corte C (uso real)	57,701 unidades, etiqueta oculta
Prueba de recorte	59,491 filas comparadas, 0 diferencias
Mejor modelo (validación, 5 folds)	Logística L2 (λ=100): precisión promedio 0.4129 ± 0.0112 (línea base constante: 0.1193; mejor línea base: 0.3892)

Dónde está cada script

Punto	**Script**
1.1	flows/pipeline.py --desde-cero (comando único), proyecto2/conteos.py y proyecto2/conteos-esperados.csv
1.2	proyecto2/poblacion.py (población, etiqueta y costo de la unidad)
1.3	proyecto2/features.py (features con la fecha de corte como parámetro), proyecto2/auditoria.py (prueba de recorte) y proyecto2/test_features.py (pruebas)
1.4	proyecto2/modelos.py (líneas base, logística L1, L2 y Elastic Net, validación cruzada)
Todos	proyecto2/config.py (cortes, ventanas y semilla) y proyecto2/bitacora.md

1.1 Datos nuevos desde cero

Los datos del Proyecto 2 reemplazan a los del Proyecto 1, no se agregan. Por eso el flujo ahora puede vaciar el entorno solo: borra el lake, el warehouse y lo que compila dbt. No tenemos snapshots de dbt ni tablas incrementales, así que no hay nada más que vaciar. No cambiamos la arquitectura: es el mismo flujo de Prefect del Proyecto 1 (Bronze por las tres vías, dbt para staging, Silver y Gold) con las tareas nuevas del Proyecto 2 agregadas al final.

Comando único

```
.venv\Scripts\python.exe flows\pipeline.py --desde-cero
```

Ese comando vacía el entorno, carga los 9 archivos a Bronze, corre dbt con sus 115 pruebas, revisa que Gold no lea Bronze, calcula las features de los tres cortes, entrena los modelos de esta fase y guarda los conteos y tiempos. No necesita ningún paso manual: los seeds los carga dbt build y las tablas se crean solas. La corrida oficial de este informe la hicimos con .venv\Scripts\python.exe flows\pipeline.py --desde-cero --auditoria, que además corre la prueba de recorte de la sección 1.3 (unos 2 minutos más); por eso aparece en la tabla de tiempos.

Tabla de conteos

Archivo	Filas esperadas	Filas cargadas	Diferencia
aerometro_boardings.csv	387,053	387,053	0
am_estaciones.csv	14	14	0
cdc_padron_usuarios.csv	77,636	77,636	0
metroriel_viajes.jsonl	590,844	590,844	0
mr_estaciones.csv	22	22	0
tm_estaciones.csv	104	104	0
transmetro_validaciones.csv	713,786	713,786	0
transurbano_transacciones.csv	1,635,788	1,635,788	0
tu_paradas.csv	328	328	0
Total	**3,405,575**	**3,405,575**	**0**

Todos los archivos cuadran exactamente con los conteos esperados que vienen en el LEEME.txt de los datos, así que no hay diferencias que explicar. Los conteos se toman de control.ingest_log, que compara las filas del archivo crudo con las que quedaron en Bronze.

Tiempos por etapa

Etapa	Segundos
Vaciar lake, warehouse y dbt/target	0.48
Bronze: batch (catálogos, MetroRiel y Transurbano)	26.42
Bronze: CDC del padrón	0.89
Bronze: streaming (Transmetro y Aerómetro, simulado)	100.97
Vaciar staging	0.10
dbt build (staging, Silver, Gold y 115 pruebas)	151.55
Verificar que Gold no lee Bronze	0.03
Conteos de la corrida	0.21
Exportar entregables del Proyecto 1	0.72
Población, etiqueta y costo de la unidad (cortes A, B y C)	2.10
Features de los cortes A, B y C	8.99
Líneas base y logística (validación cruzada)	167.63
Prueba de recorte (flujo completo con datos recortados)	122.76
Total	**582.85**

1.2 Etiqueta, población y unidad

Definición de la etiqueta

La etiqueta vale 1 si la persona no tiene ningún abordaje válido, en ningún operador, en los 30 días que empiezan a las 00:00 (hora local) de la fecha de corte, y 0 si tiene al menos uno.

Unidad elegida: la persona

Cada fila es una persona (usuario_sk). En el Proyecto 1 ya resolvimos la identidad: las tarjetas de Transmetro, Transurbano y MetroRiel se enlazan por su número (TC-00012345, 0000012345 y MR0012345 son la misma persona) y Aerómetro queda aparte porque su llave es un hash que no trae el número. Usar la persona evita contar como abandono a alguien que solo cambió de operador, que es lo que más le importa a la Agencia: que la persona siga en la red, no una tarjeta en particular. El costo es Aerómetro: sus usuarios son personas aparte, y si se pasan a otro sistema parecen un abandono. Lo medimos abajo.

Población

Entran a un corte las personas con al menos 2 abordajes válidos en los 28 días antes del corte y sin un DELETE del CDC aplicado antes del corte en ninguna de sus tarjetas. Los abordajes válidos son los de silver_abordajes: no incluyen lo que está en cuarentena ni los pagos rechazados de Transurbano.

Corte	Fecha de corte	Población	Positivos	Tasa positiva	Cuenta
A	2026-07-01	59,491	7,095	0.1193	7,095 / 59,491 = 0.1193
B	2026-07-31	58,113	6,387	0.1099	6,387 / 58,113 = 0.1099

El corte C (30 de agosto) tiene 57,701 personas en la población. Su etiqueta queda vacía porque los datos terminan el 29 de agosto y todavía no existen los 30 días siguientes.

Costo de la unidad

Medida	Corte A	Corte B
Positivos (persona)	7,095	6,387
Positivos que son personas solo de Aerómetro (no enlazadas)	1,460	1,360
De esos, siguieron viajando con otra tarjeta (falso abandono)	414	449
Costo de la unidad persona	414 de 7,095 = 0.058	449 de 6,387 = 0.070
Si la unidad fuera la tarjeta: positivos	11,423	10,523
Tarjetas positivas cuya persona siguió viajando con otra tarjeta	2,404 de 11,423 = 0.210	2,400 de 10,523 = 0.228

Cómo lo contamos: tomamos los positivos que son personas solo de Aerómetro y buscamos si su hash corresponde a alguna persona de los otros sistemas con la regla md5('am' + número) que revela el script generador. Esa regla no la usamos para construir la unidad (una Agencia real no la tendría), solo para medir el costo, igual que el análisis de sensibilidad del Proyecto 1. Para comparar, también contamos qué pasaría si la unidad fuera la tarjeta: muchas más tarjetas aparecen como abandono aunque la persona siguió viajando con otra. Por eso nos quedamos con la persona: su costo es mucho menor.

1.3 Features a la fecha de corte y auditoría de fuga

Partimos de la tabla de features por usuario del Proyecto 1 (sección 2.3) y la ajustamos para entrenar un modelo: la función recibe la fecha de corte como parámetro, todas las ventanas miden 28 días o menos en los tres cortes, solo usa registros válidos (lo que está en cuarentena no entra) y produce una fila por persona en cada corte, todo en una misma tabla con la fecha de corte como columna.

Tabla de features

La tabla completa está en entregables/proyecto2/features-cortes.csv y en el warehouse como proyecto2.features_cortes. Tiene las columnas corte, unidad_id, fecha_corte, las 23 features y etiqueta. unidad_id es el seudónimo de la persona (usuario_sk), no la tarjeta.

Corte	Fecha de corte	Filas (una por unidad)	Etiqueta
A	2026-07-01	59,491	calculada
B	2026-07-31	58,113	calculada
C	2026-08-30	57,701	vacía (oculta)

unidad_id (seudónimo)	fecha_corte	viajes_28d	días desde último	tendencia_14d	etiqueta
000363bac594…	2026-07-01	41	17	-41	0
000484e3a5ac…	2026-07-01	6	1	6	0
00067dd7d3fd…	2026-07-01	3	7	3	0

Diccionario de features

Feature	Qué mide	Ventana	Tabla de Silver
viajes_28d	Cantidad de abordajes válidos	28 días	silver_abordajes
viajes_14d	Cantidad de abordajes válidos recientes	14 días	silver_abordajes
viajes_7d	Cantidad de abordajes válidos de la última semana	7 días	silver_abordajes
tendencia_14d	Viajes de los últimos 14 días menos los de los 14 anteriores	28 días	silver_abordajes
dias_activos_28d	Días distintos con al menos un viaje	28 días	silver_abordajes
semanas_activas_28d	Semanas de siete días hacia atrás con al menos un viaje	28 días	silver_abordajes
dias_desde_ultimo_viaje	Días entre el último viaje de la ventana y el corte	28 días	silver_abordajes
dias_desde_primer_viaje_28d	Días entre el primer viaje de la ventana y el corte	28 días	silver_abordajes
modos_distintos_28d	Operadores distintos utilizados	28 días	silver_abordajes
prop_transmetro	Proporción de viajes en Transmetro	28 días	silver_abordajes
prop_transurbano	Proporción de viajes en Transurbano	28 días	silver_abordajes
prop_metroriel	Proporción de viajes en MetroRiel	28 días	silver_abordajes
prop_aerometro	Proporción de viajes en Aerómetro	28 días	silver_abordajes
prop_hora_pico_28d	Proporción en días laborales no festivos de 6 a 8 o de 17 a 19 horas	28 días	silver_abordajes; festivos_gt
prop_fin_semana_28d	Proporción de viajes en	28 días	silver_abordajes

	sábado o domingo		
gasto_q_28d	Suma de tarifas en quetzales usando centavos enteros	28 días	silver_abordajes
tarifa_promedio_q	Tarifa media en quetzales redondeada a cuatro decimales	28 días	silver_abordajes
estaciones_distintas_28d	Estaciones o paradas distintas utilizadas	28 días	silver_abordajes
zonas_distintas_28d	Zonas distintas de abordaje	28 días	silver_abordajes
pagos_rechazados_28d	Intentos de pago rechazados por Transurbano	28 días	silver_transacciones_rechazadas_tu
en_padron	Existe una versión del padrón vigente justo antes del corte	estado al corte	silver_padron_scd2
perfil	Perfil de la versión vigente más reciente o sin_padron	estado al corte	silver_padron_scd2
cambios_padron_28d	Actualizaciones aplicadas al padrón en la ventana	28 días	silver_padron_scd2

Ninguna ventana pasa de 28 días. en_padron y perfil no son ventanas: son el estado del padrón justo antes del corte, que se lee del SCD Tipo 2 del Proyecto 1 (la versión vigente en ese momento), así que no usan cambios posteriores.

Prueba de recorte

Calculamos las features del corte A dos veces: con los datos completos y con los archivos crudos recortados al 30 de junio. La segunda vez no recortamos solo Silver, sino los 9 archivos: corrimos todo el flujo desde cero en un warehouse aparte. Así, si cualquier paso (Bronze, cuarentena, identidad, CDC o features) usara datos posteriores al corte, se notaría. Además agregamos un control negativo: una feature con fuga a propósito (el perfil del padrón al final de los datos, en vez del perfil al corte), para comprobar que la prueba sí la detecta.

Medida	**Resultado**
Filas comparadas (corte A)	59,491
Features comparadas	23
Filas que están en una sola de las dos tablas	0
Diferencias antes de corregir (suma en punto flotante)	46
Diferencias con centavos enteros	0
Features que fallaron	ninguna (antes: tarifa_promedio_q)
Control negativo (perfil al final de los datos, con fuga a propósito)	13,913 cambios reales: la prueba sí detecta la fuga

La primera vez la prueba encontró 46 diferencias, todas en tarifa_promedio_q. No era una fuga temporal: el promedio se redondeaba a 4 decimales sobre una suma en punto flotante, y DuckDB suma en paralelo en distinto orden, así que los valores que caían justo a la mitad (por ejemplo 59.10 / 48 = 1.23125) a veces daban 1.2312 y a veces 1.2313. Por eso el número exacto puede cambiar un poco entre corridas y máquinas. Lo corregimos calculando en centavos enteros y la prueba quedó en 0 diferencias. El resultado antes de corregir está en entregables/proyecto2/1.3-prueba-recorte-antes.json (se repite con python proyecto2/auditoria.py --version-anterior) y la prueba final con python proyecto2/auditoria.py.

En el control negativo contamos solo los cambios reales de perfil (13,913 personas): si una persona no tiene perfil ni en el corte ni al final, lo tomamos como igual y no como diferencia. Que la prueba encuentre esos cambios demuestra que sí detecta una feature con fuga.

1.4 Líneas base y regresión logística regularizada

Métrica elegida

Usamos la precisión promedio (average precision, el área bajo la curva precisión-recall). La campaña no alcanza para todos: la Agencia va a contactar a los usuarios con más riesgo, así que importa qué tan bien quedan arriba los que de verdad se van. Además los positivos son pocos (12%), y en ese caso la precisión promedio es más exigente que el ROC-AUC, que se ve bien aunque el modelo se equivoque mucho entre los primeros. No depende de un umbral, que elegimos hasta la Fase 2 con los costos. Reportamos el ROC-AUC solo como complemento.

Validación

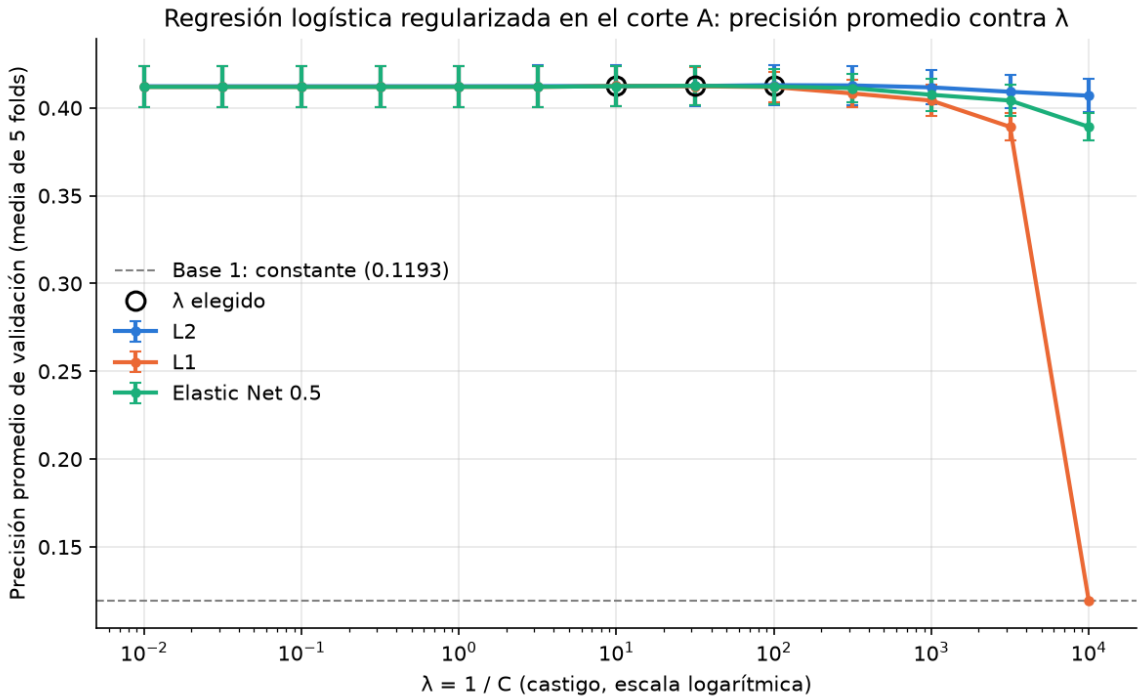
Validación cruzada estratificada de 5 folds solo con el corte A, con semilla fija (2026). Todos los modelos, incluidas las líneas base, se evalúan con los mismos 5 folds (guardados en entregables/proyecto2/folds-corte-a.csv). El escalado y la codificación del perfil van dentro de un Pipeline, así que se ajustan solo con los datos de entrenamiento de cada fold.

Tabla de modelos

Modelo	Precisión promedio (media ± desv.)	ROC-AUC (media ± desv.)
Base 1: constante (tasa del corte A)	0.1193 ± 0.0000	0.5000 ± 0.0000
Base 2: días desde el último viaje	0.3892 ± 0.0078	0.8229 ± 0.0042
Base 3: menos viajes en 28 días	0.2541 ± 0.0063	0.7426 ± 0.0064
Logística L2 (λ=100)	0.4129 ± 0.0112	0.8312 ± 0.0041
Logística L1 (λ=10)	0.4124 ± 0.0114	0.8315 ± 0.0039
Logística Elastic Net 0.5 (λ=31.62)	0.4126 ± 0.0113	0.8316 ± 0.0039

La Base 1 le da a todos la misma probabilidad, la tasa positiva del corte A (7,095 / 59,491 = 0.1193), así que su precisión promedio es justo esa tasa: no sabe ordenar a nadie. La Base 2 ordena por días desde el último viaje, sin entrenar nada: elegimos esa feature porque quien lleva más días sin viajar ya está dejando la red, y es la que más separa a los que se van (mediana de 11 días contra 2). Sola ya llega a 0.389. La Base 3 ordena por quién hizo menos viajes y queda más abajo (0.254). Las tres logísticas mejoran a la mejor línea base (0.413 contra 0.389, unas 2 desviaciones estándar) y quedan prácticamente empatadas entre sí, porque las diferencias entre castigos son mucho menores que la desviación entre folds.

Métrica contra λ



La curva es casi plana hasta λ = 100: con 59,491 filas y 23 features hay datos de sobra y castigar poco o nada da casi lo mismo. Cuando λ crece mucho, el castigo aplasta los coeficientes y la métrica cae; con L1 a λ = 10,000 todos los coeficientes quedan en 0 y el modelo vuelve a la línea base constante (0.1193). Elegimos el λ con la mejor precisión promedio de cada castigo: L2 con λ = 100 (C = 0.01), L1 con λ = 10 y Elastic Net con λ = 31.6. El mejor de los tres es L2, por muy poco.

Coeficientes

Coeficientes de la logística L2 elegida (λ = 100), entrenada con todo el corte A. Como las features numéricas están escaladas, el tamaño del coeficiente se puede comparar entre ellas: es cuánto cambia el logaritmo de las probabilidades por cada desviación estándar. Un coeficiente positivo empuja hacia el abandono.

Feature	Coeficiente	Interpretación
dias_desde_ultimo_viaje	0.690	Es el que más empuja: cuantos más días lleva sin viajar antes del corte, más probable es que no vuelva. Tiene sentido: el abandono casi siempre empieza con un silencio.
pagos_rechazados_28d	0.182	Los pagos rechazados en Transurbano (saldo insuficiente o tarjeta inválida) suben el riesgo: son fricción en el cobro, y quien no puede pagar deja de viajar.
perfil_general	0.149	Comparado con los demás perfiles, el perfil general del padrón se va más (17.5% contra 11-12%). No tiene un descuento que lo ate a la red, como el estudiante o el adulto mayor.
prop_aerometro	0.127	Quien viaja más en Aerómetro aparece con más riesgo. En parte es real y en parte es el costo de la unidad: Aerómetro no se enlaza con los otros sistemas, y si esa persona se pasa a otro modo parece que se fue.
viajes_14d	0.071	Sale positivo, pero no significa que viajar más aumente el riesgo. Está muy correlacionado con viajes_7d, viajes_28d y dias_activos_28d, que ya restan mucho: a igual cantidad de viajes en la última semana y de días activos, tener más viajes

		concentrados en las dos semanas da un pequeño ajuste. Por sí sola, más viajes = menos riesgo.

Los que más empujan hacia quedarse (coeficientes negativos) son: no estar en el padrón (perfil sin_padron, -0.59), viajes en la última semana (-0.58), días activos en 28 días (-0.55), perfil adulto mayor (-0.26) y tarifa promedio (-0.23). Todo coincide con la intuición: quien viajó hace poco y muchos días distintos sigue en la red.

Rendimiento medido

Etapa	Segundos
Carga desde cero (Bronze + dbt + exportación)	281.4
Población y etiqueta (3 cortes)	2.1
Cálculo de features (3 cortes)	9.0
Entrenamiento y validación cruzada (1.4)	167.6
Prueba de recorte (flujo completo con datos recortados)	122.7

Lo que sigue en la Fase 2: el ensemble con los mismos folds, el umbral según los costos de la campaña, la única evaluación en el corte B, la segmentación con KMeans y PCA, y predecir.py con las predicciones del corte C.